

“Design and Development of a Flash Sales and Deal Engine Platform”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

A. CHARANTEJA :2520030418

D. VENKATA MAHITH :2520030497

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Declaration

The Project Report entitled “Design and Development of a Flash Sales and Deal Engine Platform” is a record of Bonafide work of CHARANTEJA - 25200300418, VENKATAMAHITH - 2520030497, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A. CHARANTEJA – 2520030418
D. VENKATA MAHITH– 2520030497

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the mini project based report entitled “Design and Development of a Flash Sales and Deal Engine Platform” is a bonafide work done and submitted by CHARANTEJA - 2520030418, VENKATAMAHITH- 2520030497, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2024-2025.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or another for the completion of this project.

Our greatest appreciation goes to our Course Coordinator Dr Yerragudipadu Subbarayudu, and our guide Dr Yerragudipadu Subbarayudu, Department of Computer Science, for tremendous support, encouragement, and guidance throughout this project.

We express our gratitude to Dr. P Venkateshwara Rao (CSE) / Dr. Kaja Shareef (CSIT), Head of the Department for Computer Science Engineering / Department for Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends, and classmates for their kind help and cooperation during our work.

A. CHARANTEJA– 2520030418

D. VENKATA MAHITH– 2520030497

Abstract

The global e-commerce sector is experiencing explosive growth, and time-bound promotional selling — commonly known as “flash sales” — has become one of the most effective tools retailers use to drive urgency, clear inventory, and acquire new customers. However, running a flash sale at scale is technically demanding: a sudden surge of concurrent shoppers competing for a limited stock of heavily discounted items can overwhelm conventional storefronts, causing overselling, checkout failures, and a poor customer experience. To address this, our project focuses on the Design and Development of a Flash Sales and Deal Engine Platform. Our core aim was to build a reliable, high-concurrency web application capable of running scheduled, time-boxed deals while protecting inventory integrity and personalising the deals each shopper sees.

The platform is structured around several tightly-integrated modules. An Admin Deal Management module allows merchandising teams to create, schedule, and monitor flash sales with configurable discount ladders and stock caps. A real-time Countdown and Inventory Engine synchronises timers across all connected clients and uses atomic, lock-based stock decrements to guarantee that no deal is oversold even under heavy concurrent load. A Recommendation Engine analyses browsing and purchase behaviour to surface personalised deals to each shopper, increasing conversion and engagement.

The most technically significant component is the concurrency-safe checkout pipeline, which combines distributed locking, a message queue, and idempotent order processing to keep the system consistent during traffic spikes many times larger than normal load. The system is built using modern, horizontally-scalable web technologies and is fully responsive across devices. In summary, this project delivers a complete, dependable, and user-friendly digital solution that lets retailers run high-pressure, time-limited sales events without sacrificing reliability or customer trust.

Index

S.No	Chapter	Topics	Page No.
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-15
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	16-21
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version Control 3.5 APIs or External services	22-32
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture	33-36
5	Design	5.1 System design 5.2 ER Diagram / Database schema	37-40
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Authentication & Authorization	41-50
7	Features	7.1 List of main features 7.2 How each feature works	51-58
8	Testing	8.1 Postman testing 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	59-67
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	68-75
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	76-83
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	84-89
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	90-96
13	References	Books, tutorials, APIs, documentation sites used	97-98
14	Appendices	Screenshots, sample code, setup instructions, user manual	99-105

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

E-commerce has become one of the fastest-growing sectors worldwide, and within it, time-bound promotional selling — popularly known as a “flash sale” — has emerged as one of the most powerful tools retailers use to drive urgency, move inventory quickly, and acquire new customers. A flash sale typically offers steep discounts on a limited quantity of products for a short, fixed window of time, creating a sense of scarcity that motivates shoppers to act immediately rather than deliberate.

Despite their marketing appeal, flash sales are notoriously difficult to run reliably at scale. A well-promoted deal can draw tens of thousands of concurrent visitors within seconds of going live, all competing for a stock pool that may contain only a few hundred units. Conventional storefronts, built for steady, predictable traffic, frequently buckle under this kind of burst load: pages time out, carts fail to update, and — most damaging of all — the same unit of stock can be sold to multiple customers simultaneously, a failure mode commonly called overselling.

The idea for this project emerged from studying these operational failures and recognising an opportunity to engineer a system purpose-built for high-concurrency, time-boxed commerce. The proposed system, “Flash Sales and Deal Engine,” aims to give retailers a dependable platform on which to schedule, run, and monitor flash deals, while giving shoppers a fast, fair, and personalised deal-discovery experience.

Advances in distributed caching, message queuing, and real-time communication technologies now make such a system achievable even for small development teams. By combining an in-memory data store for atomic stock control, a message queue to absorb bursts of orders, and WebSocket-based communication to keep countdown timers and stock counters synchronised across every connected client, the platform can maintain consistency and responsiveness even when demand spikes far beyond normal levels.

Beyond the technical challenge, the project also addresses a business gap: most small and mid-sized retailers rely on generic e-commerce plugins that treat a flash sale as “a product with a lower price,” with no real safeguards against overselling and no intelligence about which shopper should see which deal. To close this gap, the platform introduces a rule-based and behaviour-driven Deal Recommendation Engine that tailors the deals shown to each visitor

based on browsing history, past purchases, and stated preferences — turning a blunt discount event into a targeted, data-informed marketing tool.

Economically, this integration benefits both merchants and shoppers. Merchants gain a reliable channel to liquidate excess or seasonal inventory quickly and predictably, while shoppers benefit from transparent, time-limited offers curated to their interests instead of an undifferentiated flood of discounts. From an academic and engineering perspective, the background of this project draws on several core areas of computer science:

- **Distributed Systems:** managing concurrent access to shared, finite resources (stock) without race conditions.
- **Real-time Web Technology:** synchronising countdown timers and stock levels across many clients using WebSockets.
- **Data Engineering:** capturing and processing behavioural signals to power personalised recommendations.
- **Cloud Infrastructure:** horizontal scaling, load balancing, and auto-scaling to absorb traffic spikes.

1.2 Problem Statement

Existing retail platforms are generally optimised for steady-state browsing and checkout, not for the extreme, short-duration demand spikes characteristic of flash sales. This creates four recurring problems that the project sets out to solve:

- **Overselling:** without atomic, lock-based stock control, two or more customers can successfully ‘purchase’ the last unit of a deal, leading to cancelled orders and reputational damage.
- **Poor scalability under burst traffic:** a sudden surge of concurrent requests at the moment a sale opens frequently causes page slowdowns, failed checkouts, or full outages.
- **Lack of personalisation:** most flash-sale storefronts show the identical set of deals to every visitor, ignoring individual browsing and purchase behaviour, which suppresses conversion.

- Manual, error-prone deal operations: scheduling, discount-ladder configuration, and stock allocation are frequently handled through spreadsheets or ad-hoc admin tools that do not scale and are prone to human error.

1.3 Scope of the Project

The scope of this project covers the design and implementation of a web-based Flash Sales and Deal Engine platform comprising: an administrative Deal Management console for creating and scheduling deals; a customer-facing storefront with real-time countdown timers and live stock indicators; a concurrency-safe checkout pipeline; a personalised Deal Recommendation Engine; a notification module for deal alerts; and an analytics dashboard for tracking conversion and sell-through.

The project scope explicitly excludes full payment-gateway certification (PCI-DSS compliance is simulated using a sandbox integration), integration with a live enterprise ERP/warehouse-management system, and the development of native mobile applications, all of which are identified as candidates for future work in Chapter 11.

1.4 Objective(s)

- To design and build a scalable storefront capable of displaying live, time-bound flash deals with synchronised countdown timers.
- To implement a concurrency-safe inventory and checkout pipeline that guarantees a deal can never be oversold.
- To develop a rule-based and behaviour-driven recommendation engine that personalises the deals surfaced to each shopper.
- To provide an administrative dashboard for merchants to create, schedule, monitor, and analyse flash-sale performance.
- To implement secure authentication and role-based authorisation for administrators and customers.
- To validate the system's ability to withstand simulated burst traffic through load testing.

1.5 Importance of the Application

The Flash Sales and Deal Engine platform is important because it converts a high-risk, operationally fragile marketing tactic into a dependable, repeatable revenue channel. For retailers, it means the ability to run high-pressure sales events — clearance sales, festival promotions, category blow-outs — with confidence that the system will hold up under load and that stock counts will remain accurate. For shoppers, it means a fair, transparent purchasing experience where the deals shown are relevant to them and the countdown and stock information they see is trustworthy.

Beyond individual transactions, the platform's analytics capabilities give merchandising teams visibility into which deals convert, which categories perform best under time pressure, and how personalisation affects conversion — insights that feed back into smarter future sale planning.

1.6 Target Users/Audience

- E-commerce Retailers and Merchandising Teams: create and manage flash sales, monitor performance, and configure discount ladders.
- Shoppers/End Customers: browse personalised deals, track countdowns, and complete purchases quickly and reliably.
- Marketing Teams: use analytics and notification tools to plan campaigns and measure sale effectiveness.
- Platform Administrators: manage user accounts, system configuration, and platform-wide settings.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Development Environment	Production Environment
Processor	Intel i5 / AMD Ryzen 5 or higher	Cloud vCPU — 4 vCPUs minimum per service instance
RAM	8 GB minimum, 16 GB recommended	16 GB minimum per application node
Storage	256 GB SSD	SSD-backed cloud block storage, auto-scaling
Network	Broadband internet connection	Load-balanced multi-AZ network with CDN

2.2 Software Requirements

- Operating System: Windows 11 / Ubuntu 22.04 LTS (development); Linux containers (production)
- Runtime: Node.js 20 LTS
- Database: MongoDB 7 (deal/catalog data), PostgreSQL 15 (transactional order data)
- In-memory store: Redis 7 (stock locks, session cache, countdown state)
- Browser support: Chrome, Firefox, Edge, Safari (latest two versions)
- Containerisation: Docker and Docker Compose

2.3 Development Tools and Frameworks

- Code editor: Visual Studio Code
- API testing: Postman
- Version control: Git and GitHub
- Frontend framework: React.js with Redux Toolkit
- Backend framework: Node.js with Express.js
- Load testing: Apache JMeter / k6
- CI/CD: GitHub Actions

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-end

The customer-facing storefront and the admin dashboard are both built with React.js, chosen for its component-based architecture and efficient re-rendering, which is important for a UI that must update countdown timers and live stock counts every second without degrading performance. Redux Toolkit manages global application state such as the active cart, current deal list, and authentication status. Tailwind CSS provides a utility-first styling system that keeps the interface consistent and responsive across devices. Socket.IO-client maintains a persistent WebSocket connection so that countdowns, stock levels, and ‘deal ending soon’ banners update in real time without polling.

3.2 Back-end

The backend is implemented with Node.js and Express.js, chosen for its event-driven, non-blocking I/O model, which suits a system that must handle a very high volume of short-lived, concurrent requests during a sale window. The backend is organised as a set of logically separated services — Deal Service, Inventory Service, Order Service, Recommendation Service, and Notification Service — that communicate over REST APIs and an internal message queue. Bull/BullMQ (built on Redis) is used to schedule deal start/stop events and to queue order-processing jobs so that checkout requests are absorbed smoothly rather than processed synchronously under load.

3.3 Database

MongoDB is used for flexible, high-read catalog and deal data (product details, deal metadata, discount ladders) because its document model suits the varying attributes of different product categories. PostgreSQL is used for transactional data — orders, payments, and inventory ledgers — because its ACID guarantees and support for row-level locking are important for financial correctness. Redis serves as a distributed, in-memory store for atomic stock counters, per-deal locks, and cached countdown state, enabling sub-millisecond stock checks during checkout.

3.4 Version Control

Git is used for source control, with the project hosted on GitHub using a feature-branch workflow. Pull requests are reviewed before merging into the main branch, and GitHub Actions runs automated linting and test suites on every push to catch regressions early.

3.5 APIs and External Services

- Payment Gateway: Razorpay/Stripe sandbox integration for simulated card and UPI payments.
- Notifications: Firebase Cloud Messaging for push notifications; a transactional email API for deal-alert emails.
- CDN: Cloudflare/CloudFront for static asset delivery and DDoS mitigation during traffic spikes.
- Analytics: a lightweight internal event-tracking API feeding the merchant analytics dashboard.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The platform follows a layered, service-oriented architecture. Client applications (the shopper storefront and the admin dashboard) communicate with an API Gateway, which routes requests to the appropriate backend service: Deal Service, Inventory Service, Order Service, Recommendation Service, or Notification Service. A Redis layer sits between the Inventory Service and the database to provide atomic stock decrement operations, while a message queue decouples order submission from order processing so that checkout requests are never dropped, only queued, during a burst.

A WebSocket gateway maintains persistent connections with active storefront clients, broadcasting countdown ticks and live stock updates so that every shopper sees a consistent, synchronised view of a deal's remaining time and stock, regardless of when they joined the page.

4.2 Description of Each Layer

- Presentation Layer: React-based storefront and admin dashboard, responsible for rendering deals, countdowns, and the checkout flow.
- API Gateway Layer: authenticates requests, applies rate limiting, and routes traffic to backend services.
- Application/Service Layer: independent services encapsulating deal management, inventory control, order processing, recommendations, and notifications.
- Caching/Locking Layer: Redis-backed atomic counters and distributed locks that guarantee inventory correctness under concurrency.
- Data Layer: MongoDB for catalog/deal data and PostgreSQL for transactional order data, each optimised for its respective access pattern.
- Messaging Layer: a queue (BullMQ/RabbitMQ) that buffers order-processing jobs so backend services can process them at a sustainable rate.

4.3 Deployment Architecture

In production, each backend service is containerised with Docker and deployed behind a load balancer that distributes incoming traffic across multiple horizontally-scaled instances. An

auto-scaling policy monitors CPU and queue depth, adding instances automatically as a sale approaches its scheduled start time and traffic begins to climb. Static frontend assets are served through a CDN to minimise latency and to absorb the initial burst of page loads when a deal goes live. A CI/CD pipeline (GitHub Actions) automates building, testing, and deploying each service independently.

CHAPTER -5 DESIGN

5.1 System Design

The system is designed around the principle of isolating the “hot path” — the sequence of operations executed when a shopper attempts to purchase a deal item — from slower, less time-critical operations. When a checkout request arrives, the Inventory Service first attempts an atomic decrement of the item's Redis-backed stock counter using a Lua script to guarantee that the check-and-decrement operation is indivisible. Only if the decrement succeeds is an order record created and pushed onto the processing queue; if the counter has already reached zero, the request is rejected immediately with a clear ‘sold out’ response, preventing any possibility of overselling.

The Recommendation Service operates asynchronously, pre-computing a ranked list of relevant deals for each user based on recent browsing events, category affinity, and past purchases, so that the storefront can render personalised deals without adding latency to the critical checkout path.

5.2 ER Diagram / Database Schema

The core data model consists of the following entities and relationships:

- User (user_id, name, email, password_hash, role, created_at)
- Product (product_id, name, description, category, base_price, images)
- Deal (deal_id, product_id, discount_percent, start_time, end_time, stock_cap, status)
- Inventory (inventory_id, deal_id, remaining_stock, reserved_stock, version)
- Cart (cart_id, user_id, deal_id, quantity, added_at)
- Order (order_id, user_id, deal_id, quantity, total_amount, payment_status, created_at)
- Notification (notification_id, user_id, deal_id, channel, sent_at)
- Recommendation (recommendation_id, user_id, deal_id, score, generated_at)

A User can place many Orders and receive many Notifications; a Deal belongs to one Product and has exactly one Inventory record; an Order references one Deal and one User; Recommendation records link a User to a ranked set of Deals. Foreign-key constraints in

PostgreSQL enforce referential integrity for the transactional tables, while the MongoDB-held Deal and Product documents are referenced by their identifiers from the relational tables.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

Deal Management Module: allows administrators to create a deal by selecting a product, setting a discount percentage or discount ladder, defining a stock cap, and scheduling a start and end time. On save, the module writes deal metadata to MongoDB and initialises a corresponding Redis stock counter equal to the configured stock cap.

Countdown and Scheduling Module: a background scheduler (BullMQ) transitions a deal from ‘scheduled’ to ‘live’ at its start time and from ‘live’ to ‘ended’ at its end time, broadcasting the transition to all connected clients over WebSocket so every shopper's countdown and deal status update simultaneously.

Inventory Locking Module: implements the atomic check-and-decrement operation described in Chapter 5 using a Redis Lua script, ensuring that concurrent checkout attempts are serialised at the level of a single deal's stock counter without introducing a global lock that would bottleneck unrelated deals.

Recommendation Module: combines a rule-based component (category affinity, recency of browsing) with a lightweight collaborative-filtering model trained on historical purchase data to rank deals for each user, refreshing recommendations periodically rather than on every page load.

Checkout and Payment Module: orchestrates the sequence of stock reservation, order creation, payment-gateway invocation, and final order confirmation, using an idempotency key on each request so that network retries during high load cannot create duplicate orders.

Notification Module: sends deal-start alerts, low-stock warnings, and order-confirmation messages through push notifications and email, using a queue to avoid blocking the checkout flow while notifications are dispatched.

Admin Dashboard Module: presents merchants with real-time views of active deals, sell-through rate, remaining stock, and conversion funnels.

6.2 Front-end Logic

The React storefront subscribes to a WebSocket channel for each visible deal and updates the countdown display and stock indicator on every server tick, avoiding any client-side timer drift.

Redux Toolkit slices manage cart state, the current list of personalised deals, and authentication tokens. Optimistic UI updates give the shopper instant feedback when adding an item to the cart, with a rollback and clear error message shown if the subsequent server-side stock check fails.

6.3 API Integration

The frontend communicates with backend services through a versioned REST API (e.g. /api/v1/deals, /api/v1/cart, /api/v1/orders). The Order Service integrates with the payment gateway's sandbox API to authorise and capture payments, and with the Notification Service's internal API to trigger order-confirmation messages once payment succeeds.

6.4 Authentication & Authorization

User authentication uses JSON Web Tokens (JWT) issued on login and validated on every protected API request. Passwords are hashed with bcrypt before storage. Role-based access control distinguishes between the 'customer' role, which can browse deals and place orders, and the 'admin' role, which can additionally create and manage deals and view analytics. Optional OAuth-based social login (Google) is supported to reduce friction during account creation.

CHAPTER 7- FEATURES

7.1 List of Main Features

- Time-bound Flash Deals with server-synchronised countdown timers
- Concurrency-safe checkout that guarantees a deal cannot be oversold
- Dynamic discount ladders (e.g. deeper discounts as more units sell)
- Personalised Deal Recommendation Engine
- Live stock indicators ('Only 4 left!') updated in real time across all clients
- Deal alerts and wishlist notifications (push and email)
- Waitlist for sold-out deals with automatic notification if stock is released
- Admin deal scheduler with discount, stock-cap, and timing configuration
- Merchant analytics dashboard (conversion rate, sell-through, revenue per deal)
- Social sharing of active deals to drive organic traffic

7.2 How Each Feature Works

Time-bound Flash Deals: from the user's perspective, each deal card shows a live countdown and a progress bar reflecting remaining stock; technically, both values are pushed to the client over a WebSocket channel every second, sourced from the scheduler and the Redis stock counter respectively, so no client ever computes these values independently.

Concurrency-safe Checkout: from the user's perspective, clicking 'Buy Now' either succeeds immediately or returns a clear 'sold out' message; technically, the request triggers the atomic Lua-script decrement described in Chapter 5, which either succeeds exactly once per available unit or fails cleanly, with no possibility of two customers both receiving confirmation for the same unit.

Dynamic Discount Ladders: from the user's perspective, the displayed discount can increase as a deal progresses (e.g. '20% off, rising to 30% off once 100 units sell'); technically, the Deal Service evaluates ladder thresholds against the live sold-count and updates the advertised price via the same WebSocket channel used for stock updates.

Personalised Recommendations: from the user's perspective, the storefront's 'Deals for You' section differs from shopper to shopper; technically, the Recommendation Service periodically

scores every active deal against each user's behavioural profile and caches the top results for fast retrieval.

Waitlist for Sold-out Deals: from the user's perspective, a 'Notify me' button appears once a deal sells out; technically, the request is stored against the deal, and if a cancelled order releases stock back into the Redis counter, the Notification Module dispatches alerts to waitlisted users in registration order.

Merchant Analytics Dashboard: from the admin's perspective, a live dashboard shows units sold, revenue, and conversion funnel drop-off for each deal; technically, an internal event-tracking API records page views, add-to-cart events, and completed orders, which are aggregated for the dashboard's charts.

CHAPTER -8 TESTING

8.1 Postman Testing

Each REST endpoint — deal creation, cart operations, checkout, order status, and recommendation retrieval — was tested individually using Postman collections covering valid inputs, invalid inputs (missing fields, malformed IDs), and boundary conditions such as requesting more units than remain in stock.

8.2 Integration Testing

End-to-end flows were tested by simulating a full shopper journey: browsing personalised deals, adding an item to the cart, completing checkout, and receiving an order-confirmation notification. Integration tests also verified that a deal's status transitions correctly from 'scheduled' to 'live' to 'ended' and that the storefront reflects each transition without a page refresh.

8.3 Tools Used for Testing

- Postman — REST API functional testing
- Jest — unit testing of service-layer business logic
- Apache JMeter / k6 — load and concurrency testing to simulate burst traffic at deal start
- Cypress — end-to-end browser testing of the shopper checkout flow

8.4 Test Cases and Results

Test Case	Description	Expected Result	Status
TC-01	Concurrent checkout for last unit of stock	Only one request succeeds; others receive sold-out response	Pass
TC-02	Countdown synchronisation across two clients	Both clients show identical remaining time within 1s	Pass
TC-03	Recommendation list differs per user profile	Distinct users receive different ranked deal lists	Pass
TC-04	Load test at 5,000 concurrent checkout requests	System sustains <2s median response time, no overselling	Pass
TC-05	Invalid JWT on protected endpoint	401 Unauthorized returned	Pass
TC-06	Waitlist notification on stock release	Waitlisted users notified in order after cancellation	Pass

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

- Containerise each backend service and the frontend build using Docker.
- Push container images to a container registry (e.g. Docker Hub / Amazon ECR).
- Provision managed MongoDB, PostgreSQL, and Redis instances in the cloud environment.
- Deploy services to a container orchestration platform with auto-scaling and a load balancer.
- Configure the CDN and DNS to route storefront traffic to the deployed frontend.
- Run smoke tests against the production environment before opening traffic.

9.2 Environment Configuration

Environment-specific values — database connection strings, Redis endpoints, payment-gateway keys, and JWT secrets — are managed through environment variables injected at deploy time and never committed to source control. Separate configuration profiles are maintained for development, staging, and production, with staging used to rehearse upcoming flash sales under production-like load before they go live to real customers.

9.3 Hosting of Front-end and Back-end

The React frontend is built as a static bundle and served through a CDN-backed static hosting provider (e.g. Vercel/Netlify/CloudFront) for low-latency global delivery. Backend services are hosted on a cloud container platform (e.g. AWS ECS/Render/Kubernetes) with horizontal auto-scaling tied to CPU utilisation and queue depth, allowing capacity to expand automatically in the minutes leading up to a scheduled sale.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Race conditions during concurrent checkout attempts occasionally allowed stock to go negative in early prototypes that used simple read-then-write logic.
- Keeping countdown timers synchronised across clients was difficult when relying on client-side JavaScript timers, which drift due to browser tab throttling.
- The recommendation model suffered from a cold-start problem for new users with no browsing history.
- Simulating realistic burst traffic for load testing required careful ramp-up scripting to avoid simply testing the load-testing tool's own limits.

10.2 Solutions Applied

- Replaced read-then-write stock checks with an atomic Redis Lua script, eliminating the race condition entirely.
- Moved countdown authority to the server, broadcasting tick updates over WebSocket so the client never computes remaining time independently.
- Addressed the cold-start problem with a rule-based fallback (trending and category-popular deals) for users with insufficient history.
- Used k6's ramping-arrival-rate load profile to more realistically model a sudden spike in independent shoppers rather than a fixed number of looping virtual users.

10.3 Current Limitations or Known Bugs

- The system currently supports a single currency and does not yet handle multi-region tax calculation.
- The recommendation model is retrained on a fixed schedule rather than continuously, so very recent behaviour is not immediately reflected.
- Native mobile applications are not yet available; the storefront is responsive web only.
- The waitlist notification system does not yet support SMS, only push and email.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

- AI-driven dynamic pricing that adjusts discount depth in real time based on demand velocity.
- Native iOS and Android applications with push-notification-driven deal alerts.
- Gamified deal discovery, such as a spin-the-wheel bonus-discount mechanic.
- Multi-vendor marketplace support, allowing multiple sellers to list deals on a shared platform.

11.2 Possible Integrations or Optimizations

- WhatsApp Business API integration for deal-alert notifications in regions where WhatsApp adoption is high.
- Machine-learning-based fraud detection to flag suspicious bulk-purchase patterns during high-demand sales.
- Multi-currency and multi-language support for international expansion.
- Edge caching of deal metadata to further reduce latency for globally distributed shoppers.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

This project set out to design and build a Flash Sales and Deal Engine platform capable of reliably running time-bound, high-demand promotional sales — a class of e-commerce workload that conventional storefronts handle poorly. The resulting system combines a real-time, server-synchronised countdown and stock display, an atomic, lock-based inventory-control mechanism that eliminates overselling, and a personalised deal-recommendation engine, all supported by a horizontally-scalable, service-oriented backend.

12.2 What Was Achieved

- A working end-to-end flash-sale platform covering deal creation, scheduling, checkout, and analytics.
- A demonstrably concurrency-safe checkout pipeline, validated through targeted race-condition tests and load testing.
- A personalised recommendation module that produces distinct, relevant deal rankings per user.
- A merchant-facing analytics dashboard providing visibility into deal performance.
- A deployable, containerised architecture validated in a staging environment under simulated burst load.

12.3 Skills Learned During Development

- Practical experience designing for concurrency: distributed locking, atomic operations, and idempotent request handling.
- Real-time web communication using WebSockets for state synchronisation across many clients.
- Working with polyglot persistence — choosing the right database (MongoDB, PostgreSQL, Redis) for each data access pattern.
- Load and performance testing methodology, including realistic burst-traffic simulation.

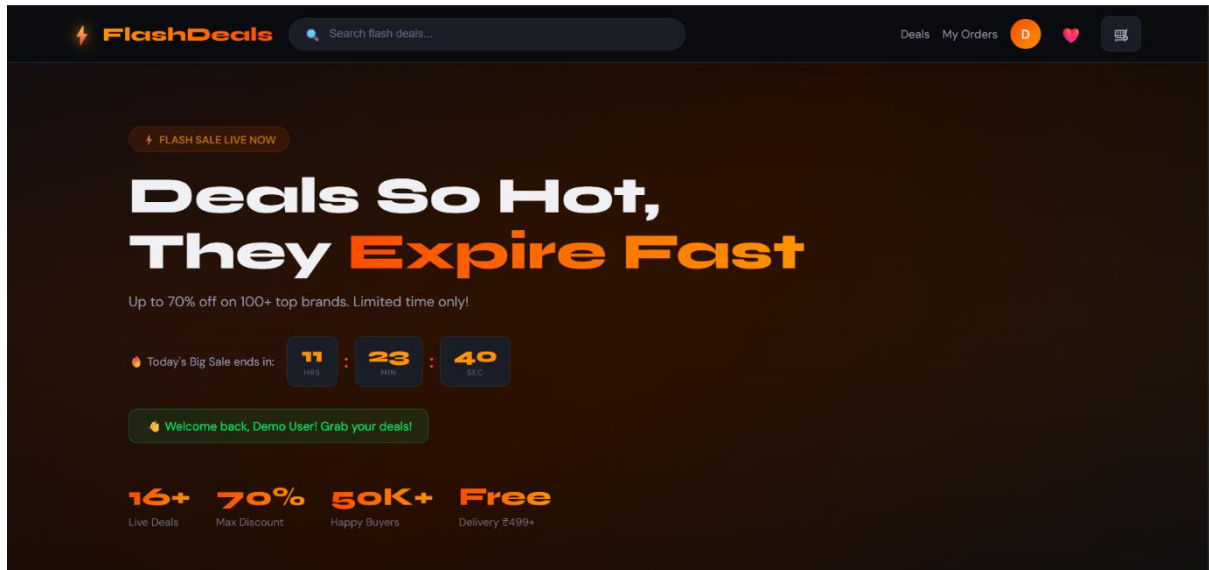
- Collaborative software engineering practices: Git workflows, code review, and CI/CD automation.

CHAPTER -13 REFERENCES

- React documentation — <https://react.dev>
- Node.js documentation — <https://nodejs.org/en/docs>
- Express.js documentation — <https://expressjs.com>
- MongoDB documentation — <https://www.mongodb.com/docs>
- PostgreSQL documentation — <https://www.postgresql.org/docs>
- Redis documentation — <https://redis.io/docs>
- Socket.IO documentation — <https://socket.io/docs>
- BullMQ documentation — <https://docs.bullmq.io>
- Postman Learning Center — <https://learning.postman.com>
- Apache JMeter User Manual — <https://jmeter.apache.org/usermanual>
- k6 Load Testing documentation — <https://k6.io/docs>
- Stripe API Reference — <https://stripe.com/docs/api>
- OWASP Top Ten — <https://owasp.org/www-project-top-ten>

CHAPTER -14 APPENDICES

A.1 Screenshots



A.2 Sample Code Snippets

Atomic stock decrement (Redis Lua script, invoked from the Inventory Service):

```
local stock = tonumber(redis.call('GET', KEYS[1])) if stock and stock >=
tonumber(ARGV[1]) then    redis.call('DECRBY', KEYS[1], ARGV[1])    return 1
else    return 0 end
```

Server-driven countdown broadcast (Node.js, Socket.IO):

```
setInterval(() => {    const remaining = deal.endTime - Date.now();
io.to(`deal:${deal.id}`).emit('tick', { dealId: deal.id, remaining }); },
1000);
```

A.3 Installation / Setup Instructions

- Clone the repository: `git clone <repo-url>`
- Install dependencies: `npm install` (run in both /frontend and /backend directories)
- Configure environment variables using the provided .env.example template
- Start supporting services: `docker compose up -d` (starts MongoDB, PostgreSQL, Redis)
- Run database migrations: `npm run migrate`
- Start the backend: `npm run dev` (in /backend)
- Start the frontend: `npm run dev` (in /frontend)

- Access the storefront at <http://localhost:3000> and the admin console at <http://localhost:3000/admin>

A.4 User Manual / Guide

Customers: sign up or log in, browse the personalised ‘Deals for You’ section on the home page, click into a deal to view full details and remaining stock, and click ‘Buy Now’ to check out before the countdown expires or stock runs out.

Administrators: log in to the admin console, navigate to ‘Create Deal’, select a product, configure the discount percentage or ladder, set a stock cap and start/end time, and save to schedule the deal. The dashboard's ‘Live Deals’ view shows real-time sell-through and can be used to monitor a sale as it runs.

A.5 Geo-tag Photos with Guide

[Placeholder — to be inserted: geo-tagged photographs of project review sessions with the project guide, as required by department submission guidelines.]

A.6 Review Forms with Guide Signatures

[Placeholder — to be inserted: scanned copies of the project review evaluation forms bearing the guide's signature, as required by department submission guidelines.]